

III RTOS - Queues

Contreras R Orlando^{1,2*} and Lara H Kevin^{1,2*}

³Department of Electronic Systems, UAA, Av. Universidad 940,
Aguascalientes, 20131, Aguascalientes, México.

*Corresponding author(s). E-mail(s): [al348390,al464376](mailto:al348390@edu.uaa.mx)@edu.uaa.mx;

Abstract

The objective of this lab practice is make multiple tasks in the RTOS framework and implement a queue to handle the data through tasks to communicate between them.

Keywords: C, ARM, Queues, RTOS, Task, Priorities

1 Introduction

In this practice, we will handle multiple tasks that happen at the same time. This by doing RTOS (Real Time Operating System) tasks whose allow to run in concurrency, apparently at the same time, but actually swaps between tasks so fast so we can perceive that this is parallel, this is called concurrency. The queues are one of the most useful tools in the FreeRtos libraries in embedded systems, we can send data between tasks, and send control signals in all the code.

2 Objectives

- Correct use of queues to handle values.
- Defining priorities to avoid timing errors.
- Correct use of GPIO.

3 Method

3.1 Pseudo-code

Here is the pseudo-code that represents the algorithm, it is also included the configuration as masks or

Algorithm 1 Button Task (vTaskButton)

Input: None

Output: Button state monitoring and queue messaging

```
1 lastState  $\leftarrow$  1 // Initial button state (not pressed)
2 while 1 do
3   if  $((GPIOE.IDR) \wedge (1 \ll 8)) = 0$  then
4     actualState  $\leftarrow$  0 // Button pressed
5   else
6     actualState  $\leftarrow$  1 // Button released
7   if lastState = 1  $\wedge$  actualState = 0 then
8     pdMS_TO_TICKS(20) // Debounce delay
9     xPeriodQueue, actualState, pdMS_TO_TICKS(20) xPeriodQueue,
      actualState, pdMS_TO_TICKS(20) // Send two messages
10  lastState  $\leftarrow$  actualState pdMS_TO_TICKS(50) // Polling interval
```

Algorithm 2 LED Button Task (vTaskLedButton)

Input: None

Output: LED toggling based on button state

```
11 while 1 do
12   waitsTicks  $\leftarrow$  pdMS_TO_TICKS(10) // 10ms timeout
13   if xQueueReceive(xPeriodQueue, recievedQueue, waitsTicks) = pdPASS then
14     if recievedQueue = 0 then
15       GPIOA.ODR  $\leftarrow$  GPIOA.ODR  $\oplus$  (1  $\ll$  5) // Toggle PA5 LED
```

Algorithm 3 LED Frequency Task (vTaskLedFreq)

Input: None

Output: Frequency-modulated LED toggling

```
16 vec[0]  $\leftarrow$  500, vec[1]  $\leftarrow$  250 // Frequency periods in ms
17 pos  $\leftarrow$  0 // Current frequency index
18 while 1 do
19   waitsTicks  $\leftarrow$  pdMS_TO_TICKS(vec[pos]) // Dynamic timeout
20   if xQueueReceive(xPeriodQueue, recievedQueue, waitsTicks) = pdPASS then
21     if recievedQueue = 0 then
22       pos  $\leftarrow$  pos  $\oplus$  1 // Toggle frequency index
23   GPIOA.ODR  $\leftarrow$  GPIOA.ODR  $\oplus$  (1  $\ll$  6) // Toggle PA6 LED
```

4 Results and Discussion

4.1 Code

Here is the code that represents the previous pseudo-code

Code: queue.c

```
int main(void){
confRCC();
confGPIO();
xPeriodQueue = xQueueCreate(2,sizeof(uint32_t));
xTaskCreate(vTaskButton,"LED-TASK2",100,NULL,1,NULL);
xTaskCreate(vTaskLedButton,"LED-TASKk",100,NULL,2,NULL);
xTaskCreate(vTaskLedFreq,"LED-TASK",100,NULL,2,NULL);
vTaskStartScheduler();
while(1);
}

void confRCC(void){
RCC->AHB1ENR |= (1 << 0) | (1 << 4);
}

void confGPIO(void){
    GPIOA->MODER |= (5 << 10); // PA5 Y PA6 Output
    GPIOE->PUPDR |= (1 << 16); // PE8 Button
}

void vTaskLedButton (void *pvPARAMETERS){

    uint32_t recievedQueue;
    while(1){
        TickType_t waitsTicks = (pdMS_TO_TICKS (10));
        if(xQueueReceive(xPeriodQueue,&recievedQueue , waitsTicks) == pdPASS)
        {
            if (recievedQueue == 0)
                GPIOA->ODR ^= (1 << 5);}
        }
    }

void vTaskLedFreq (void *pvPARAMETERS){
    uint16_t vec[2] = {500,250};
    uint8_t pos =0;

    uint32_t recievedQueue;
    while(1){
        TickType_t waitsTicks = (pdMS_TO_TICKS (vec[pos]));
        if(xQueueReceive(xPeriodQueue,&recievedQueue , waitsTicks) == pdPASS)
        {
            if(recievedQueue == 0) pos ^= 1;
        }
        GPIOA->ODR ^= (1 << 6);
    }
}
```

```

void vTaskButton(void *pvPARAMETERS){
    uint32_t lastState = 1;

    while(1){
        uint32_t actualState;
        if(!((GPIOE->IDR) & (1 << 8))) actualState = 0;
        else actualState = 1;
        if(lastState ==1 && actualState == 0 )
        {
            vTaskDelay(pdMS_TO_TICKS(20));
            xQueueSend(xPeriodQueue,&actualState ,pdMS_TO_TICKS(20));
            xQueueSend(xPeriodQueue,&actualState ,pdMS_TO_TICKS(20));
        }
        lastState= actualState;
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}

```

4.2 Wiring Diagram

Here is our circuit implementation; we configure the button as pull up as shown in [Fig 1]

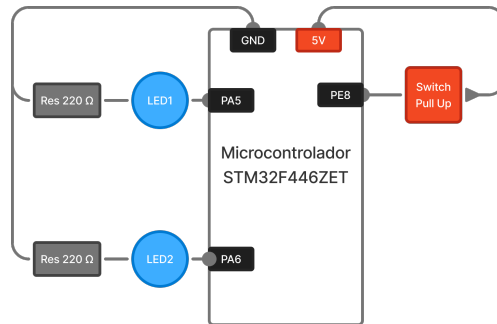


Fig. 1: Wiring Diagram

5 Conclusion

In this practice, we can observe the correct functionality of FreeRTOS queues and tasks to correctly change the LED toggle frequency. The Queues are valuable tools in embedded systems coding to create a better workflow on multitask programming allowing us to handle separate tasks and interruptions, this could be useful in tasks where the timing is crucial and the data handling between tasks.